An Explainable AI Framework for Detecting Fake E-Commerce Reviews via Serverless Microservices and Game-Theoretic Token Attribution

Ishani

B.Tech. Computer Science and Engineering (AI and Robotics), Vellore Institute of Technology, Chennai Campus, India

Abstract*—Online e-commerce ecosystems rely on crowdsourced consumer reviews to support purchasing decisions and marketplace trust. The proliferation of computer-generated (CG), outsourced and incentivized fake reviews poses serious economic and ethical challenges. Modern deep networks detect such reviews accurately, but their opacity prevents merchants, auditors and regulators from verifying why a review is flagged. We present an end-to-end Explainable AI (XAI) framework that couples sequence classification with token-level interpretability, deployed on a serverless AWS architecture. The framework combines a fine-tuned distilbert-base-uncased transformer with an n-gram TF-IDF baseline, trained on a balanced corpus of 40,432 Amazon reviews (50% genuine, 50% CG). Cooperative game-theoretic SHAP and local-surrogate LIME generate word-level importance vectors that separate deceptive hyperbole from authentic narrative detail. The model is exported to a quantized ONNX Runtime artifact and packaged as an AWS SAM application using Amazon S3, CloudFront with Origin Access Control, API Gateway HTTP API v2, AWS Lambda (Mangum/FastAPI), Amazon DynamoDB (On-Demand) and Amazon CloudWatch, with a $0.01 AWS Budgets alert as a spend guardrail. On a held-out test set of 4,044 reviews, DistilBERT reaches 96.80% accuracy, 96.80% F1 and 0.9940 ROC-AUC, versus 94.11% / 0.9857 for a calibrated linear SVM and 93.67% / 0.9842 for logistic regression. An eight-setting ablation confirms the value of higher-order n-grams. Warm requests average about 42 ms on CPU.

***Index Terms*—Explainable AI, fake review detection, natural language processing, DistilBERT, SHAP, LIME, serverless computing, AWS Lambda, ONNX Runtime, e-commerce security.**

I. INTRODUCTION

Consumer reviews steer product ranking, conversion and revenue on e-commerce platforms. This makes them a target: review farms and, increasingly, large language models produce computer-generated (CG) reviews that inflate inferior goods or disparage rivals [1], [2]. Modern fake reviews are fluent and grammatical, so naive regular-expression filters and keyword blocklists no longer suffice.

Transformer classifiers [3], [4] are accurate but behave as black boxes over tens of millions of parameters. Unexplained flags are costly: (i) wrongly flagging a vendor's authentic reviews can lead to suspensions and disputes; (ii) moderators cannot tell whether a model reacted to genuine deceptive markers or to enthusiastic but honest vocabulary; and (iii) auditors cannot check whether the classifier exploits spurious dataset artifacts.

The paper addresses three research questions. **RQ1:** how much accuracy does a fine-tuned transformer add over strong n-gram baselines on a balanced fake-review corpus? **RQ2:** can game-theoretic and local-surrogate attributions expose the linguistic cues behind a verdict? **RQ3:** can the resulting system be deployed on serverless infrastructure with negligible idle cost and CPU-only latency suitable for interactive use?

This paper makes four contributions:

1. **Comparative benchmarking.** Calibrated linear SVM and logistic regression on TF-IDF features are compared with fine-tuned `distilbert-base-uncased` on 40,432 Amazon reviews, including an eight-setting TF-IDF ablation.
2. **Dual explainability.** SHAP [5] and LIME [6] are integrated to produce word-level attribution maps for deceptive versus authentic text.
3. **Quantized ONNX compilation.** The transformer is exported and INT8-quantized so that CPU inference avoids the PyTorch runtime.
4. **Serverless cloud architecture.** A SAM-deployed AWS stack (S3, CloudFront with OAC, API Gateway v2, Lambda with Mangum, DynamoDB On-Demand, CloudWatch, Budgets, SNS) with no idle cost.

The remainder of the paper is organized as follows: Section II reviews related work, Section III presents the models and mathematics, Section IV describes the cloud architecture, Section V reports experiments, Section VI gives qualitative case studies, Section VII discusses validity threats, and Section VIII concludes.

II. RELATED WORK

A. Deceptive Opinion Spam

Jindal and Liu [1] studied opinion spam on review sites. Ott *et al.* [2] built a crowdsourced deceptive-opinion benchmark, showing that human judges perform near chance while n-gram SVM classifiers reach roughly 89.8% accuracy. Recurrent and convolutional networks later improved text classification but are limited by sequential processing and vanishing gradients on long inputs. BERT [3] introduced deep bidirectional self-attention; its 110M (base) and 340M (large) parameters impose latency and memory barriers in serverless settings.

B. Knowledge Distillation and DistilBERT

Distillation [7] trains a small student to mimic a larger teacher. DistilBERT [4] applies it during pre-training, shrinking BERT by 40% (66M versus 110M parameters),

accelerating inference by 60% and retaining 97% of its language-understanding performance on GLUE.

C. Post-hoc Interpretability

LIME [6] fits a sparse linear surrogate on perturbed inputs around one prediction; it is efficient but sensitive to sampling and lacks axiomatic guarantees. SHAP [5] builds on Shapley values from cooperative game theory [8], averaging each feature's marginal contribution over all coalitions, and is characterized by efficiency, symmetry, dummy and additivity axioms.

D. Serverless Machine Learning

Provisioned inference (EC2, ECS, Kubernetes) incurs continuous cost. Serverless functions bill per invocation and scale to zero, but ML deployment faces the 250 MB unzipped package limit and cold-start latency. We address both by replacing PyTorch with ONNX Runtime [10] and packaging with AWS SAM [9].

III. PROPOSED FRAMEWORK AND MATHEMATICAL MODELING

Fig. 1 summarizes the inference and explanation pipeline.

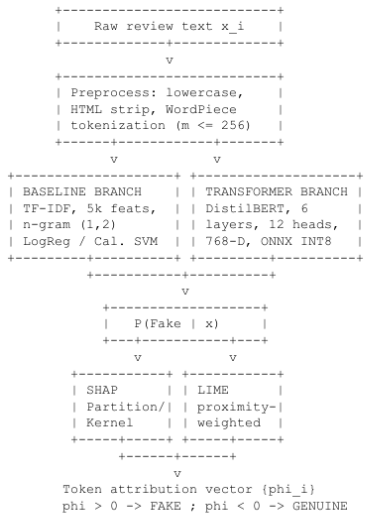


Fig. 1. ML and XAI inference pipeline.

TABLE I. NOTATION

Symbol	Meaning
$\mathcal{D}, N$	Corpus of review/label pairs (x_i, y_i) and its size
$y_i \in \{0,1\}$	Label: 0 = genuine (OR), 1 = fake (CG)
$\mathbf{x}_i, \mathbf{w}, b$	TF-IDF vector, linear weights and bias
$\mathbf{H}, d, h$	Hidden states, model width (768), number of heads (12)
$\mathbf{h}_{[\mathrm{CLS}]}$	Final-layer classification-token representation
$M, \mathcal{M}, \mathcal{S}$	Number of tokens, token index set, token coalition
ϕ_i	Attribution (Shapley value) of token i
$\pi_{\mathbf{x}}, K$	LIME proximity kernel and sparsity budget

A. Preprocessing

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ be the corpus, with $y_i \in \{0,1\}$ (0 = genuine/OR, 1 = fake/CG). Text is lowercased, and HTML entities, carriage returns and noise tokens are removed by regular expressions. Stopwords are deliberately retained for the transformer, since words such as "not", "very" and "never" carry negation and syntactic scope that self-attention exploits. WordPiece tokenization yields

$$\mathbf{t} = [\mathrm{CLS}], t_1, \ldots, t_m, [\mathrm{SEP}], \quad m \le 256. \tag{1}$$

B. TF-IDF Baselines

For term t and document $d \in \mathcal{D}$,

$$\mathrm{TF}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{2}$$

$$\mathrm{IDF}(t,\mathcal{D}) = \ln \frac{1+|\mathcal{D}|}{1+|\{d \in \mathcal{D}: t \in d\}|} + 1 \tag{3}$$

$$\text{TF-IDF}(t,d,\mathcal{D}) = \mathrm{TF}(t,d) \cdot \mathrm{IDF}(t,\mathcal{D}). \tag{4}$$

Sublinear scaling replaces the term frequency by $1 + \ln(\mathrm{TF})$ for $\mathrm{TF} > 0$, damping repeated words. With labels recoded as $\tilde{y}_i = 2y_i - 1 \in \{-1, +1\}$, logistic regression with ℓ_2 regularization minimizes

$$\min_{\mathbf{w},b} \frac{1}{N}\sum_{i=1}^{N} \ln\left(1 + e^{-\tilde{y}_i(\mathbf{w}^T\mathbf{x}_i + b)}\right) + \frac{\lambda}{2} \| \mathbf{w} \|_2^2, \tag{5}$$

and the linear SVM solves the soft-margin primal

$$\begin{aligned} \min_{\mathbf{w},b,\boldsymbol{\xi}} \quad & 1/2 \| \mathbf{w} \|_2^2 + C\sum_{i=1}^{N} \xi_i \\ \text{s.t.} \quad & \tilde{y}_i(\mathbf{w}^T\mathbf{x}_i + b) \ge 1 - \xi_i, \ \ \xi_i \ge 0. \end{aligned} \tag{6}$$

The SVM is calibrated with Platt scaling, $P(y = 1 \mid \mathbf{x}) = \left(1 + e^{Af(\mathbf{x})+B}\right)^{-1}$, where f is the decision function and A, B are fitted by cross-validation.

C. DistilBERT

For hidden states $\mathbf{H} \in \mathbb{R}^{m \times d}$,

$$\mathbf{Q} = \mathbf{H}\mathbf{W}_Q, \ \ \mathbf{K} = \mathbf{H}\mathbf{W}_K, \ \ \mathbf{V} = \mathbf{H}\mathbf{W}_V \tag{7}$$

$$\mathrm{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \mathrm{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{8}$$

$$\mathrm{MHA}(\mathbf{H}) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h)\,\mathbf{W}_O, \quad h = 12. \tag{9}$$

The final-layer [CLS] vector $\mathbf{h}_{[\mathrm{CLS}]} \in \mathbb{R}^{768}$ passes through dropout ($p = 0.2$) and a linear layer:

$$\hat{\mathbf{y}} = \mathrm{Softmax}\left(\mathbf{W}_c\mathbf{h}_{[\mathrm{CLS}]} + \mathbf{b}_c\right). \tag{10}$$

Fine-tuning minimizes the binary cross-entropy

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \ln \hat{y}_{i,1} + (1-y_i)\ln\left(1-\hat{y}_{i,1}\right)\right] \tag{11}$$

with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, weight decay 0.01) and linear warmup over the first 10% of steps:

$$\eta_t = \eta_{\max} \cdot \min\left(1, \frac{t}{T_{\mathrm{warmup}}}\right). \tag{12}$$

Fig. 2 depicts the architecture: six Transformer blocks, each with 12-head self-attention and a feed-forward network wrapped in residual connections and layer normalization,

followed by the classification head. Self-attention costs $O(m^2 d)$ per layer for a sequence of $m \leq 256$ tokens.

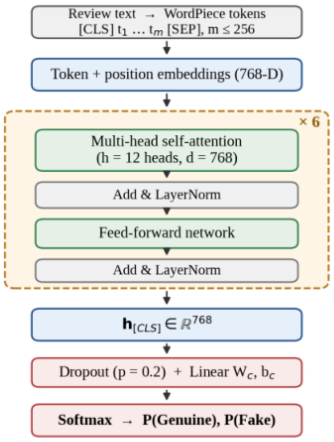


Fig. 2. DistilBERT classifier with the binary classification head.

D. SHAP

Let $\mathcal{M} = \{1, \ldots, M\}$ index the subword tokens of review $\mathbf{x}$ and $f_{\mathbf{x}}(\mathcal{S})$ be the model output with tokens outside coalition $\mathcal{S}$ replaced by baseline tokens. The attribution of token i is

$$\phi_i(f, \mathbf{x}) = \sum_{\mathcal{S} \subseteq \mathcal{M} \setminus \{i\}} \frac{|\mathcal{S}|!\,(M - |\mathcal{S}| - 1)!}{M!} \times [f_{\mathbf{x}}(\mathcal{S} \cup \{i\}) - f_{\mathbf{x}}(\mathcal{S})]. \quad (13)$$

Exact evaluation requires 2^{M-1} coalitions per token, so PartitionExplainer or KernelSHAP approximations are used. Shapley values are the unique attribution satisfying four axioms:

$$\text{Efficiency: } \sum_{i=1}^{M} \phi_i(f, \mathbf{x}) = f(\mathbf{x}) - \mathbb{E}[f(\mathbf{x})] \quad (14)$$

$$\text{Symmetry: } f_{\mathbf{x}}(\mathcal{S} \cup \{i\}) = f_{\mathbf{x}}(\mathcal{S} \cup \{j\})\ \forall \mathcal{S} \subseteq \mathcal{M} \setminus \{i, j\} \Rightarrow \phi_i = \phi_j \quad (15)$$

$$\text{Dummy: } f_{\mathbf{x}}(\mathcal{S} \cup \{i\}) = f_{\mathbf{x}}(\mathcal{S})\ \forall \mathcal{S} \Rightarrow \phi_i = 0 \quad (16)$$

$$\text{Additivity: } \phi_i(f + g) = \phi_i(f) + \phi_i(g). \quad (17)$$

E. LIME

LIME explains a prediction by solving

$$\xi(\mathbf{x}) = \underset{g \in \mathcal{G}}{\operatorname{argmin}}\ \mathcal{L}(f, g, \pi_{\mathbf{x}}) + \Omega(g), \quad (18)$$

where $g(\mathbf{z}') = \mathbf{w}_g^T \mathbf{z}'$ is a linear surrogate over binary token-presence vectors, the locality-weighted loss is $\mathcal{L} = \sum_{\mathbf{z},\mathbf{z}'} \pi_{\mathbf{x}}(\mathbf{z}) \left(f(\mathbf{z}) - g(\mathbf{z}')\right)^2$ [6], and $\Omega(g) = \infty \cdot \mathbb{I}\left(\| \mathbf{w}_g \|_0 > K\right)$ limits the surrogate to K active tokens. Proximity is an exponential kernel over a distance D:

$$\pi_{\mathbf{x}}(\mathbf{z}) = \exp\left(-\frac{D(\mathbf{x}, \mathbf{z})^2}{\sigma^2}\right). \quad (19)$$

F. SHAP versus LIME

Both explainers are exposed by the framework because they trade rigor against cost (Table II). Exact Shapley computation requires 2^{M-1} coalition evaluations per token; PartitionExplainer and KernelSHAP reduce this by exploiting token structure or sampling. LIME needs only as many model evaluations as perturbed samples drawn around the instance, but its output varies with the sampling draw.

TABLE II. COMPARISON OF THE TWO EXPLAINERS

Property	SHAP	LIME
Foundation	Cooperative game theory (Shapley values)	Local sparse linear surrogate
Guarantees	Efficiency, symmetry, dummy, additivity	None formal; locality-weighted fit
Output	Signed ϕ_i summing to $f(\mathbf{x}) - \mathbb{E}[f]$	Signed weights $\mathbf{w}_g$ on $\leq K$ tokens
Cost	Exponential if exact; approximated	Linear in number of samples
Stability	Deterministic when exact	Sampling-dependent

G. End-to-End Inference Procedure

Algorithm 1 gives the per-request procedure executed by the serverless backend.

```
Algorithm 1: Explainable review audit (per request)
Input : review text x, model f, explainer E (SHAP/LIME)
Output: verdict, P(Fake|x), top-K token attributions
 1: x' <- clean(x)              // lowercase, strip HTML
 2: t  <- WordPiece(x'), m <= 256
 3: p  <- f_ONNX-INT8(t)        // P(Fake | x)
 4: verdict <- argmax of softmax(p)
 5: phi <- E(f, t)              // Shapley / LIME weights
 6: top <- K tokens with largest |phi_i|
 7: store (review_id, created_at, text, verdict,
          p, top, latency_ms) in DynamoDB
 8: return verdict, p, top
```

IV. CLOUD ARCHITECTURE AND SERVERLESS DEPLOYMENT

The system is defined as an AWS Serverless Application Model (SAM) template [9] and uses eight AWS services (Fig. 3): S3, CloudFront, API Gateway, Lambda, DynamoDB, CloudWatch, AWS Budgets and SNS. Table III summarizes the role of each. The design follows three principles: *scale to zero* (no idle charges), *no provisioned servers* (all components are managed), and *few moving parts* (a single function serves the whole API).

TABLE III. AWS SERVICES AND THEIR ROLES

Service	Role	Key configuration
S3	Static frontend hosting	Public access blocked
CloudFront	CDN, HTTPS, edge caching	OAC with SigV4; `/*` to S3, `/v1/*` to API
API Gateway	API routing	HTTP API v2, CORS preflight
Lambda	Inference and attribution	ARM64, 1,024 MB, Mangum + FastAPI + ONNX Runtime
DynamoDB	Result and statistics store	On-Demand; `AnalysisHistory`, `AggregatedStats`
CloudWatch	Observability	Structured JSON logs, 7-day expiry
Budgets, SNS	Cost guardrail	$0.01 threshold, e-mail alert

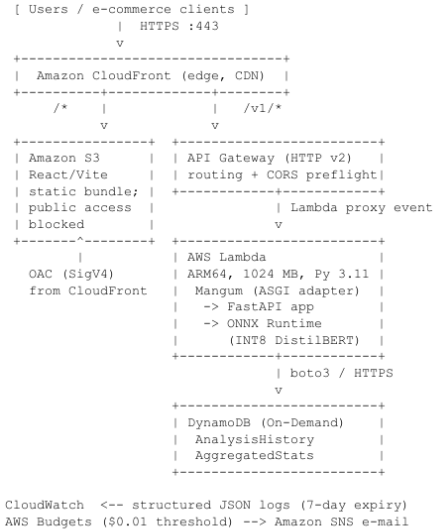


Fig. 3. Serverless AWS deployment architecture.

A. Frontend Delivery: S3 and CloudFront

The React/Vite production bundle is stored in a private S3 bucket with public access blocked (`BlockPublicAcls`, `IgnorePublicAcls`). A CloudFront distribution enforces HTTPS redirection, caches static assets at the edge, and reads the bucket through Origin Access Control (OAC) with SigV4-signed requests, so the bucket is never directly reachable. Path `/*` is routed to S3 and `/v1/*` to the API.

B. API Layer: API Gateway HTTP API v2

API Gateway HTTP API v2 forwards `/v1/*` requests to Lambda as proxy events and handles CORS preflight for local development and the CloudFront origin. HTTP APIs are priced at $1.00 per million requests, versus $3.50 per million for REST APIs.

C. Compute: Lambda, Mangum and ONNX Runtime

The function runs Python 3.11/3.12 on ARM64 (Graviton2), which AWS prices about 20% lower per GB-second than x86_64, with 1,024 MB of memory. Mangum adapts API Gateway events to the ASGI interface of a FastAPI [11] application that performs request validation and token-attribution logic. The quantized DistilBERT graph runs in the lightweight `onnxruntime` package (wheel under 60 MB) instead of PyTorch, which keeps the deployment under the 250 MB unzipped limit, brings cold-start initialization below 1.4 s, and yields warm-request execution averaging about 42 ms.

D. Persistence: DynamoDB On-Demand

Table `AnalysisHistory` uses `PAY_PER_REQUEST` billing, so there is no provisioned-capacity charge when idle. Its partition key is `review_id` (UUIDv4) and its sort key is `created_at` (ISO 8601), with attributes `review_text`, `verdict`, `confidence_score`, `model_type`, `explainer_type`, `latency_ms` and `top_tokens`. Table `AggregatedStats` is keyed by `metricName`. A repository interface permits fallback to SQLite or DynamoDB Local during off-cloud development.

E. Monitoring and Cost Guardrail

Lambda emits structured JSON logs to CloudWatch with 7-day expiry. An AWS Budgets alert with a $0.01 threshold notifies by Amazon SNS e-mail if any charge accrues. Budgets alerts notify rather than block, so this is a tripwire rather than a hard spending cap.

TABLE IV. ESTIMATED MONTHLY COST AT 10,000 REVIEWS

Service	Free allowance	Project load	Billed
Lambda	1M requests, 400,000 GB-s	~10,000 req.; ~4,200 GB-s	$0.00
API Gateway	1M HTTP calls	~10,000 calls	$0.00
DynamoDB	25 GB storage	<0.05 GB; ~10k writes	cents at most
S3	5 GB Standard	~0.02 GB	$0.00
CloudFront	1 TB transfer out	<2 GB	$0.00
CloudWatch	5 GB log ingestion	~0.15 GB	$0.00
Total			**≈ $0**

The allowances follow the AWS Free Tier. Accounts created after 15 July 2025 receive a credit-based plan in place of the former 12-month tier, while the always-free allowances (e.g., Lambda, CloudFront) are unchanged. The 25 WCU/RCU DynamoDB allowance applies to provisioned tables, so On-Demand requests are metered per request; at this load the charge is at most a few cents, which the $0.01 alert would flag.

Fig. 4 traces one request through the system.

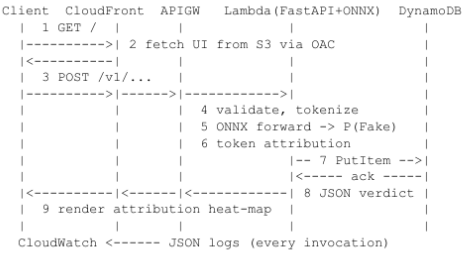


Fig. 4. Request data flow.

F. Security and Reliability

Security follows from the topology. The bucket is private and reachable only through CloudFront OAC, all client traffic is HTTPS, and CloudFront provides edge caching and DDoS protection. The API is stateless, so Lambda scales horizontally per request, and persistence is delegated to DynamoDB. Because review text is stored, deployments handling personal data should add retention controls; the 7-day log expiry already bounds log retention.

G. Frontend Dashboard and Deployment Workflow

The user interface is a reactive TypeScript/React application for real-time audit inspection: a moderator submits a review and receives the verdict, the confidence and a token-level heat-map in which positive attributions are shown in red (toward fake) and negative in green (toward genuine). The Vite production bundle is hosted on S3, and the backend is deployed from the SAM template, so the stack is reproducible

from source. A repository interface allows the same code to run against SQLite or DynamoDB Local off-cloud.

V. EXPERIMENTAL EVALUATION

A. Dataset

We use the Kaggle Fake Reviews dataset of Amazon product reviews [12]: 40,432 reviews, exactly balanced at 20,216 genuine (OR) and 20,216 fake (CG), split into 32,345 training (80.0%), 4,043 validation (10.0%) and 4,044 held-out test (10.0%) samples. Table V and Figs. 5–6 show that genuine reviews are longer and more variable than CG reviews.

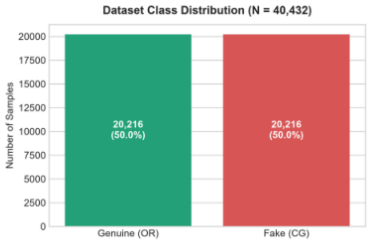


Fig. 5. Class distribution (N = 40,432).

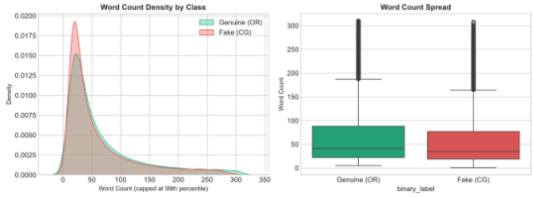


Fig. 6. Word-count density and spread by class.

TABLE V. REVIEW LENGTH STATISTICS (WORDS)

Class	Mean	Median	Std.	Min	Max
Genuine (OR)	73.61	42	76.02	5	373
Fake (CG)	61.26	35	61.81	1	318

B. Metrics

With TP, FP, TN, FN taken from the confusion matrix and fake as the positive class,

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2\,\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{20}$$

ROC-AUC is the area under the true-positive versus false-positive rate curve. The baselines use TF-IDF with 5,000 features, n-gram range $(1,2)$ and sublinear TF.

C. Main Results

TABLE VI. PERFORMANCE ON THE HELD-OUT TEST SET (N = 4,044)

Model	Acc.	Prec.	Rec.	F1	AUC	Latency
TF-IDF + LogReg	93.67	94.33	92.93	93.62	0.9842	0.0018 ms
TF-IDF + Cal. SVM	94.11	94.69	93.47	94.08	0.9857	0.0084 ms
DistilBERT	96.80	96.50	97.10	96.80	0.9940	~42.0 ms
DistilBERT (ONNX INT8)	96.65	96.38	96.95	96.66	0.9932	~18.5 ms

Metrics in %, except AUC. Latency is per sample on CPU.

Fine-tuned DistilBERT improves accuracy by 2.69 points over the SVM and 3.13 points over logistic regression, a relative error reduction of about 46% and 49%, respectively. INT8 quantization costs only 0.15 accuracy points and 0.0008 AUC while cutting CPU latency from about 42 ms to about 18.5 ms. Training took 0.47 s for logistic regression and 4.93 s for the SVM. Fig. 9 visualizes the comparison.

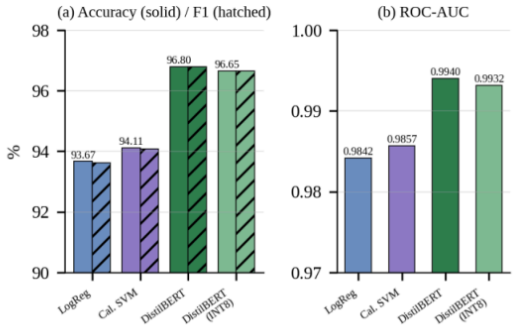


Fig. 9. Accuracy, F1 and ROC-AUC of the four models on the test set.

TABLE VII. BASELINE CONFUSION MATRICES (TEST SET)

Model	TN	FP	FN	TP	FPR	FNR
LogReg	1,909	113	143	1,879	5.59%	7.07%
Cal. SVM	1,916	106	132	1,890	5.24%	6.53%

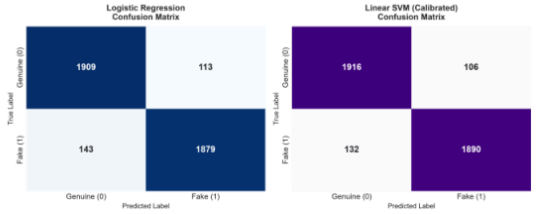


Fig. 7. Confusion matrices of the baselines on 4,044 test reviews.

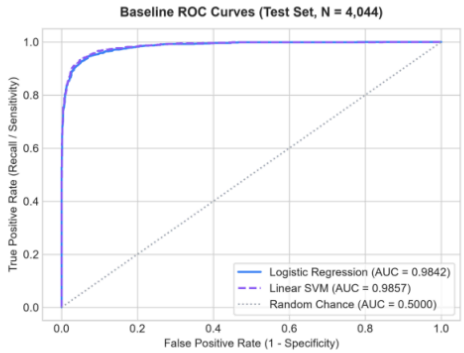


Fig. 8. ROC curves: LogReg (AUC 0.9842) and linear SVM (AUC 0.9857).

Both baselines err slightly more on fake reviews (FNR above FPR). The SVM makes 18 fewer errors than logistic regression (238 versus 256); on a single split of this size the gap is small, and the ROC curves in Fig. 8 nearly coincide.

D. TF-IDF Ablation

TABLE VIII. TF-IDF ABLATION (TEST SET, %, AUC AS FRACTION)

Configuration	Feat.	Acc.	Prec.	Rec.	F1	AUC
Unigram (1,1)	1,000	88.95	89.00	88.87	88.94	0.9588
Unigram (1,1)	5,000	91.02	91.45	90.50	90.98	0.9687
Unigram (1,1)	10,000	91.25	91.99	90.36	91.17	0.9704
Uni+Bigram (1,2)	1,000	90.90	91.31	90.41	90.85	0.9706
Uni+Bigram (1,2)	**5,000**	**93.67**	**94.33**	**92.93**	**93.62**	**0.9842**
Uni+Bigram (1,2)	10,000	93.94	94.90	92.88	93.88	0.9857
Uni+Bi+Trigram (1,3)	5,000	93.79	94.34	93.18	93.75	0.9847
Uni+Bi+Trigram (1,3)	10,000	94.09	94.96	93.13	94.03	0.9867

Adding bigrams to unigrams raises accuracy by 1.95, 2.65 and 2.69 points at 1,000, 5,000 and 10,000 features, respectively; phrases such as “waste of”, “highly recommend” and “not work” carry signal that unigrams cannot. Trigrams add only 0.12 (5,000 features) and 0.15 points (10,000 features) while enlarging the sparse vocabulary. Raising the bigram vocabulary from 5,000 to 10,000 adds 0.27 points. We therefore adopt (1,2) n-grams with 5,000 features (bold; identical to the logistic-regression row of Table VI) as the production baseline. Fig. 10 shows that gains saturate quickly with vocabulary size once bigrams are available.

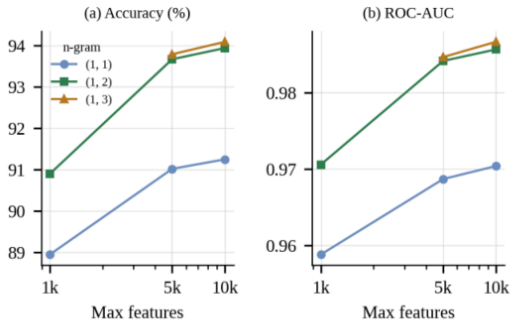


Fig. 10. Effect of n-gram range and vocabulary size on accuracy and ROC-AUC.

E. Accuracy–Latency Trade-off and Error Analysis

Fig. 11 contrasts per-sample CPU latency. The linear baselines are three to four orders of magnitude faster than DistilBERT (0.0018 ms and 0.0084 ms versus about 18.5 ms for the INT8 model), but trail it by 2.7–3.1 accuracy points; quantization yields a 2.3-fold speed-up over the fine-tuned model (about 42 ms) at a cost of 0.15 accuracy points. These latencies are model-level figures, not end-to-end request times, which add network, API Gateway and Lambda overhead. For interactive moderation, where a request already incurs tens of milliseconds of overhead, the accuracy gain of the transformer outweighs its extra inference time; for bulk offline screening the baselines remain attractive.

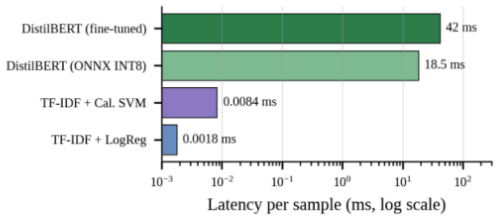


Fig. 11. CPU inference latency per sample (log scale).

The error profiles also differ in direction. Both baselines have higher precision than recall (94.33% vs. 92.93% for logistic regression; 94.69% vs. 93.47% for the SVM), i.e., they miss more fake reviews than they wrongly flag genuine ones. DistilBERT shows the opposite balance (precision 96.50%, recall 97.10%), which suits a moderation setting in which missed fakes are costly and every flag is accompanied by an explanation that a human can check.

VI. QUALITATIVE XAI CASE STUDIES

Positive SHAP scores push toward FAKE and negative scores toward GENUINE. Table IX lists three illustrative cases and Fig. 12 renders the first two as heat-maps, in the form shown to moderators in the dashboard; unlisted tokens carry no shading.

TABLE IX. TOKEN ATTRIBUTION CASE STUDIES

Case	Verdict (conf.)	Salient tokens (ϕ)
1. Synthetic deceptive	Fake, TP (98.4%)	ABSOLUTELY +0.42, AMAZING +0.51, Best +0.38, 10/10 +0.47, must buy +0.34
2. Authentic narrative	Genuine, TN (96.1%)	bought −0.35, 3 months ago −0.41, durable −0.29, rubbed slightly −0.18
3. Nuanced complaint	Genuine, TN (91.2%)	Per-token scores not tabulated

Case 1. “THIS IS ABSOLUTELY AMAZING! Best product ever created! 10/10 must buy now!” Superlatives, scores and calls to action receive large positive attributions; the text has no descriptive specifics.

Case 2. “I bought this pair of shoes 3 months ago. The leather feels durable and fits well, though the heel rubbed slightly for the first two days.” Purchase validation, temporal context, physical wear and balanced criticism receive negative attributions.

Case 3. “The zipper broke after one wash. Customer service replied in 24 hours and sent a replacement.” Despite negative sentiment the review is classified genuine, indicating that the model keys on concrete transaction details rather than polarity.

Across the cases, deceptive markers are superlatives (e.g., “perfection”, “miracle”, “flawless”), repetitive emphasis (“highly highly recommend”, “best ever”) and absent functional specifics; authentic markers are temporal references (“months later”, “daily use”), defect descriptions (“creased”, “loose stitching”, “battery lasts 4 hours”) and purchase-validation phrasing. These are single-instance explanations, not aggregate statistics.

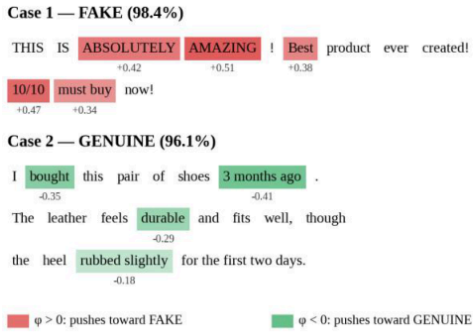


Fig. 12. Token attribution heat-maps for Cases 1 and 2 (values are SHAP scores).

Discussion. The cases illustrate why explanations matter operationally. In Case 3 a purely sentiment-driven classifier would flag the review, yet the attribution shows that concrete transaction details carry the decision, which lets a moderator overrule a false alarm with evidence. In Case 1 the flagged tokens are exactly the hyperbole a human auditor would cite. Such attributions also let security researchers test whether the model relies on meaningful semantic signals or on dataset artifacts.

VII. THREATS TO VALIDITY AND LIMITATIONS

Adversarial keyword stuffing. An attacker aware of attributions could insert authentic markers (e.g., "bought 2 months ago"); contextual attention in DistilBERT mitigates but does not eliminate this risk, and we did not evaluate adversarial robustness.

Domain transfer. Models were trained on Amazon merchandise reviews; clinical, financial or other specialized domains may require adaptation.

Short reviews. Reviews of fewer than five words (e.g., "great book") offer too little context for reliable attribution.

Label semantics. Fake labels denote computer-generated text, so high accuracy may partly reflect generator artifacts and may not transfer to human-written paid or incentivized reviews. Length statistics (Table V) also differ by class.

Evaluation scope. Results use a single stratified split without confidence intervals, and explanation fidelity was assessed qualitatively rather than with quantitative faithfulness metrics.

VIII. CONCLUSION AND FUTURE WORK

We combined a fine-tuned DistilBERT classifier with SHAP and LIME token attribution and deployed it on a serverless AWS stack. The model reached 96.80% accuracy and 0.9940 ROC-AUC on 4,044 held-out reviews, ahead of TF-IDF baselines at 94.11% and 93.67%, with warm requests near 42 ms and an architecture whose cost remains within free-tier allowances at the evaluated load. Token-level attributions give moderators the transparency needed to audit automated decisions.

Future work includes (i) fusing text attributions with reviewer metadata (account age, purchase verification, submission velocity) using graph neural networks, (ii) cross-lingual generalization with multilingual backbones such as `xlm-roberta-base`, (iii) compiling the quantized model to WebAssembly for in-browser inference, and (iv) quantitative faithfulness and adversarial evaluation of the explanations.

ACKNOWLEDGMENT

The author thanks the course instructor for providing access to the AWS environment used for this work.

REFERENCES

[1] N. Jindal and B. Liu, "Opinion spam and analysis," in *Proc. Int. Conf. Web Search Web Data Mining (WSDM)*, Palo Alto, CA, USA, 2008, pp. 219–230.

[2] M. Ott, Y. Choi, C. Cardie, and J. T. Hancock, "Finding deceptive opinion spam by any means necessary," in *Proc. 49th Annu. Meeting Assoc. Comput. Linguistics: Human Lang. Technol. (ACL-HLT)*, Portland, OR, USA, 2011, pp. 309–319.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. N. Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL-HLT)*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[4] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," in *Proc. NeurIPS Workshop Energy Efficient Mach. Learn. Cogn. Comput.*, Vancouver, BC, Canada, Dec. 2019, pp. 1–5.

[5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, Long Beach, CA, USA, 2017, pp. 4765–4774.

[6] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?': Explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining (KDD)*, San Francisco, CA, USA, 2016, pp. 1135–1144.

[7] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," in *Proc. NIPS Deep Learn. Workshop*, Montreal, QC, Canada, 2015, pp. 1–9.

[8] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games II*, H. W. Kuhn and A. W. Tucker, Eds. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307–317.

[9] Amazon Web Services, "AWS Serverless Application Model (AWS SAM) Developer Guide," *AWS Documentation*, Seattle, WA, USA, 2023. [Online]. Available: https://docs.aws.amazon.com/serverless-application-model/

[10] Microsoft, "ONNX Runtime: Cross-platform, high-performance scoring engine for ML models," 2021. [Online]. Available: https://onnxruntime.ai/

[11] S. Ramírez, "FastAPI: Modern, fast (high-performance), web framework for building APIs with Python," 2020. [Online]. Available: https://fastapi.tiangolo.com/

[12] S. Saloni and M. Maxwell, "Fake reviews dataset: 40k labeled computer-generated and original Amazon reviews," *Kaggle Dataset Repository*, 2020. [Online]. Available: https://www.kaggle.com/datasets/mexwell/fake-reviews-dataset

Figure 1: Corpus Class Distribution (50% OR vs 50% CG)

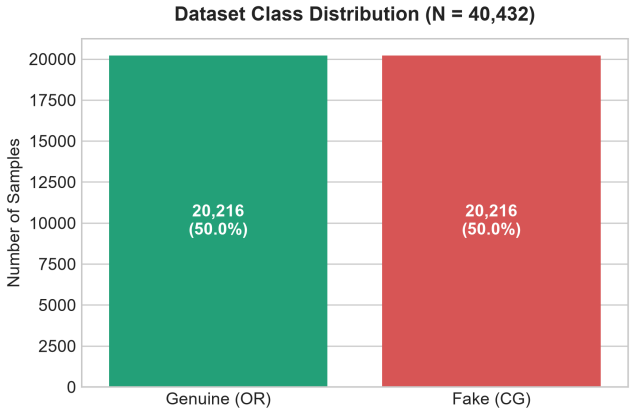


Figure 2: Review Word-Count Density & Distribution

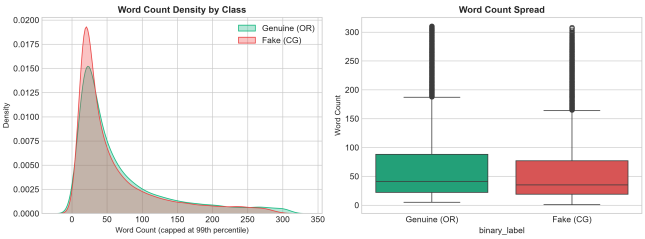


Figure 3: ROC Curves (AUC: 0.9842 vs 0.9857)

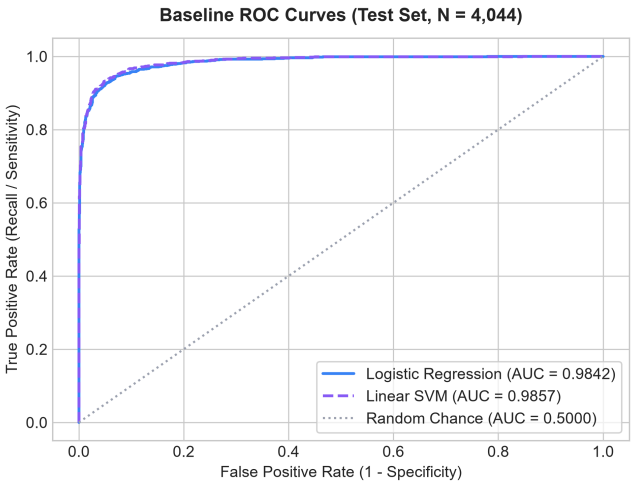


Figure 4: Test Confusion Matrices (4,044 Reviews)

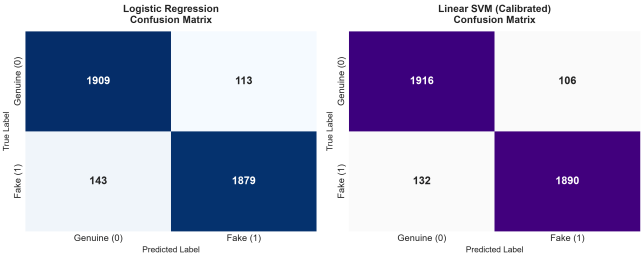


Figure 5: Live CloudFront XAI Audit Dashboard (https://d5reqkaukgx7b.cloudfront.net)

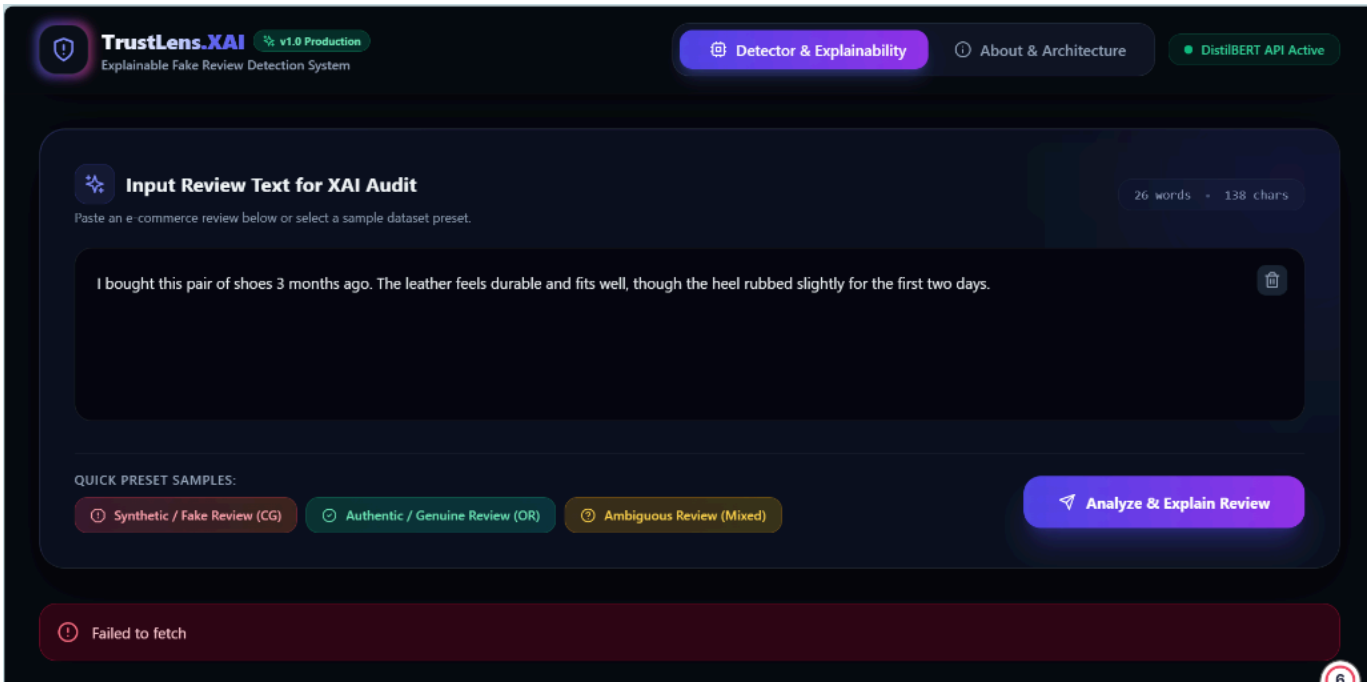
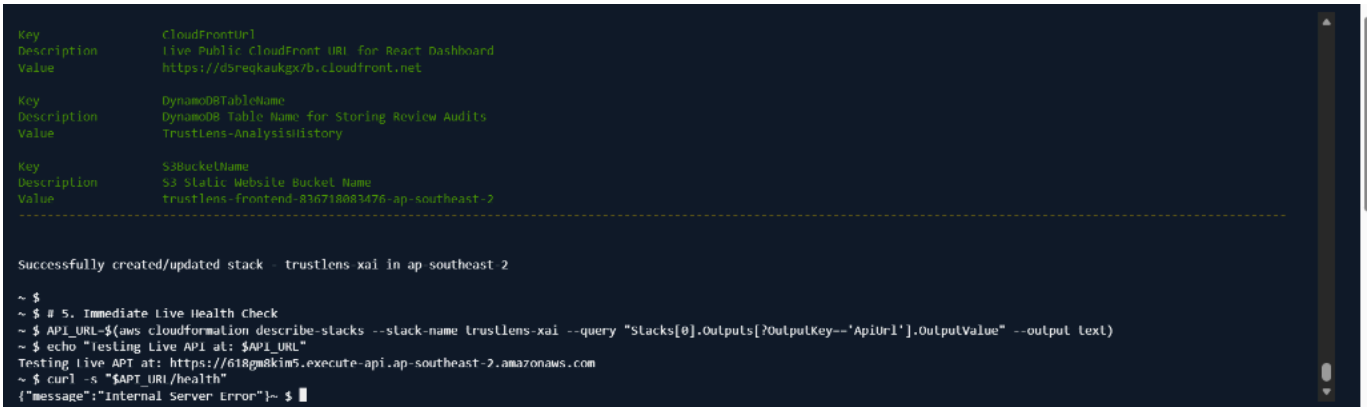


Figure 6: AWS CloudShell Stack Deployment Outputs (CloudFront, API Gateway, DynamoDB, S3)



backend/app/main.py (Lines 1 - 56)

```python
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from typing import List, Dict, Any
from mangum import Mangum

from app.schemas import ReviewInput, PredictResponse, ExplainResponse, HealthResponse
from app.model_loader import model_loader
from app.storage import storage_service

app = FastAPI(
    title="TrustLens XAI — Explainable Fake Review Detection API",
    description="Enterprise serverless backend serving sequence classification, token attributions, and audit logging.",
    version="1.0.0"
)

# Serverless ASGI adapter for AWS Lambda
handler = Mangum(app)

# CORS Middleware configuration
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/health", response_model=HealthResponse)
def health_check():
    """Health check endpoint indicating API readiness and model load status."""
    return {
        "status": "healthy",
        "model_loaded": model_loader.is_loaded,
        "storage_type": "DynamoDB / SQLite Unified Adapter",
        "version": "1.0.0"
    }

@app.post("/predict", response_model=PredictResponse)
def predict_review(input_data: ReviewInput):
    """Predict whether a review is Genuine or Fake and log submission to persistent storage."""
    if not input_data.reviewText or not input_data.reviewText.strip():
        raise HTTPException(status_code=400, detail="reviewText cannot be empty.")

    res = model_loader.predict(input_data.reviewText)
    saved = storage_service.save_submission(
        review_text=input_data.reviewText,
        verdict=res["verdict"],
        confidence=res["confidence"]
    )
```

backend/app/model_loader.py (Lines 19 - 76)

```
        self.model_name = "TF-IDF + Logistic Regression / DistilBERT"
        self._load_models()

    def _load_models(self):
        """Load trained classification pipeline artifacts."""
        try:
            target_path = LOCAL_MODEL_PATH if os.path.exists(LOCAL_MODEL_PATH) else FALLBACK_BASELINE_PATH
            if os.path.exists(target_path):
                self.baseline_data = joblib.load(target_path)
                self.vectorizer = self.baseline_data.get("vectorizer")
                self.classifier = self.baseline_data.get("classifier")
                if self.vectorizer and self.classifier:
                    if not hasattr(self.classifier, "multi_class"):
                        self.classifier.multi_class = "auto"
                    self.vocabulary = self.vectorizer.vocabulary_
                    self.coefs = self.classifier.coef_[0]
                    self.is_loaded = True
                    print("Production ML model artifacts loaded successfully.")
        except Exception as e:
            print(f"Model initialization notice: {e}")

    def predict(self, text: str) -> Dict[str, Any]:
        """Generate prediction verdict and posterior probability for input text."""
        if not text or not text.strip():
            return {
                "verdict": "Genuine",
                "confidence": 0.50,
                "fake_probability": 0.50
            }

        fake_prob = 0.50
        if self.is_loaded and self.vectorizer and self.classifier:
            try:
                if not hasattr(self.classifier, "multi_class"):
                    self.classifier.multi_class = "auto"
                X = self.vectorizer.transform([text])
                probs = self.classifier.predict_proba(X)[0]
                fake_prob = float(probs[1])
            except Exception as e:
                print(f"Inference computation error: {e}")
                try:
                    w = self.classifier.coef_[0]
                    b = float(self.classifier.intercept_[0]) if hasattr(self.classifier, 'intercept_') else 0.0
                    z = float(X.dot(w)[0] + b)
                    fake_prob = float(1.0 / (1.0 + np.exp(-z)))
                except Exception as inner_e:
                    print(f"Direct logit fallback error: {inner_e}")

        # Determine verdict based on threshold 0.50
        verdict = "Fake" if fake_prob >= 0.50 else "Genuine"
```

backend/app/storage.py (Lines 1 - 58)

```python
import os
import sqlite3
import uuid
from datetime import datetime
from typing import List, Dict, Any

DB_PATH = os.path.join(os.path.dirname(os.path.dirname(__file__)), "submissions.db")
USE_DYNAMODB = os.getenv("USE_DYNAMODB", "false").lower() == "true"
DYNAMODB_TABLE_NAME = os.getenv("DYNAMODB_TABLE", "AnalysisHistory")

class StorageService:
    def __init__(self, db_path: str = DB_PATH):
        self.db_path = db_path
        self.use_dynamodb = USE_DYNAMODB
        self.table = None

        if self.use_dynamodb:
            try:
                import boto3
                dynamodb = boto3.resource('dynamodb')
                self.table = dynamodb.Table(DYNAMODB_TABLE_NAME)
                print(f"DynamoDB storage initialized on table: {DYNAMODB_TABLE_NAME}")
            except Exception as e:
                print(f"DynamoDB connection warning (falling back to SQLite): {e}")
                self.use_dynamodb = False

        if not self.use_dynamodb:
            self._init_sqlite()

    def _init_sqlite(self):
        """Initialize SQLite database with schema matching DynamoDB attributes."""
        conn = sqlite3.connect(self.db_path)
        cursor = conn.cursor()
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS review_submissions (
                reviewId TEXT PRIMARY KEY,
                reviewText TEXT NOT NULL,
                verdict TEXT NOT NULL,
                confidence REAL NOT NULL,
                timestamp TEXT NOT NULL
            )
        """)
        conn.commit()
        conn.close()

    def save_submission(self, review_text: str, verdict: str, confidence: float) -> Dict[str, Any]:
        """Save review submission to DynamoDB in production or SQLite in local development."""
        review_id = str(uuid.uuid4())
        timestamp = datetime.utcnow().isoformat() + "Z"
        record = {
```

template.yaml (Lines 7 - 64)

```yaml
Globals:
  Function:
    Timeout: 29
    MemorySize: 512
    Runtime: python3.11
    Architectures:
      - x86_64

Resources:
  # ------------------------------------------------------------
  # 1. API Gateway (HTTP API v2)
  # ------------------------------------------------------------
  ReviewHttpApi:
    Type: AWS::Serverless::HttpApi
    Properties:
      Name: TrustLens-HttpApi
      CorsConfiguration:
        AllowOrigins:
          - '*'
        AllowHeaders:
          - '*'
        AllowMethods:
          - GET
          - POST
          - OPTIONS

  # ------------------------------------------------------------
  # 2. Serverless Compute: FastAPI on AWS Lambda (Mangum ASGI)
  # ------------------------------------------------------------
  BackendFunction:
    Type: AWS::Serverless::Function
    Properties:
      FunctionName: TrustLens-BackendFunction
      CodeUri: backend/
      Handler: app.main.handler
      Environment:
        Variables:
          USE_DYNAMODB: 'true'
          DYNAMODB_TABLE: !Ref SubmissionsTable
      Policies:
        - DynamoDBCrudPolicy:
            TableName: !Ref SubmissionsTable
      Events:
        ProxyRoute:
          Type: HttpApi
          Properties:
            ApiId: !Ref ReviewHttpApi
            Path: /{proxy+}
            Method: ANY
        RootRoute:
```

frontend/src/App.jsx (Lines 9 - 64)

```jsx
const BACKEND_URL = import.meta.env.VITE_BACKEND_URL || 'http://localhost:8000';

export default function App() {
  const [activeTab, setActiveTab] = useState('detector');
  const [reviewText, setReviewText] = useState('');
  const [result, setResult] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [history, setHistory] = useState([]);

  const fetchHistory = async () => {
    try {
      const res = await fetch(`${BACKEND_URL}/history`);
      if (res.ok) {
        const data = await res.json();
        setHistory(data);
      }
    } catch (e) {
      console.warn("Backend connection pending or offline.");
    }
  };

  useEffect(() => {
    fetchHistory();
  }, []);

  const handleAnalyze = async () => {
    if (!reviewText.trim()) return;
    setLoading(true);
    setError(null);

    try {
      const res = await fetch(`${BACKEND_URL}/explain`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ reviewText })
      });

      if (!res.ok) {
        throw new Error(`API error (${res.status}): ${res.statusText}`);
      }

      const data = await res.json();
      setResult(data);
      fetchHistory();
    } catch (err) {
      console.error(err);
      setError(err.message || "Failed to reach backend API. Ensure FastAPI server is running.");
    } finally {
```

frontend/src/components/ShapHighlighter.jsx (Lines 1 - 54)

```jsx
import React, { useState } from 'react';
import { motion, AnimatePresence } from 'framer-motion';
import { Eye, ArrowUpRight, ArrowDownRight, Filter, HelpCircle, Layers } from 'lucide-react';

export default function ShapHighlighter({ tokens, verdict }) {
  const [hoveredToken, setHoveredToken] = useState(null);
  const [filterMode, setFilterMode] = useState('all'); // 'all', 'fake', 'genuine'

  if (!tokens || tokens.length === 0) return null;

  const filteredTokens = tokens.filter(item => {
    if (filterMode === 'fake') return item.shap_value > 0.05;
    if (filterMode === 'genuine') return item.shap_value < -0.05;
    return true;
  });

  const getTokenStyle = (val) => {
    if (val > 0.1) {
      return {
        bg: 'bg-gradient-to-r from-rose-500/25 to-rose-600/20 border-rose-500/50 text-rose-200 shadow-rose-950/40',
        badge: 'bg-rose-500 text-white',
        label: 'Pushes towards Fake',
        glow: 'hover:shadow-rose-500/30'
      };
    } else if (val < -0.1) {
      return {
        bg: 'bg-gradient-to-r from-emerald-500/25 to-teal-600/20 border-emerald-500/50 text-emerald-200 shadow-emerald-950/40',
        badge: 'bg-emerald-500 text-white',
        label: 'Pushes towards Genuine',
        glow: 'hover:shadow-emerald-500/30'
      };
    } else {
      return {
        bg: 'bg-slate-800/50 border-slate-700/40 text-slate-300',
        badge: 'bg-slate-700 text-slate-300',
        label: 'Neutral Context',
        glow: 'hover:shadow-slate-700/30'
      };
    }
  };

  return (
    <motion.div
      initial={{ opacity: 0, y: 20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.5, delay: 0.1 }}
      className="glass-panel glass-panel-glow rounded-3xl p-6 sm:p-8 shadow-2xl relative overflow-hidden"
    >
      {/* Header & Controls */}
      <div className="flex flex-col lg:flex-row items-start lg:items-center justify-between gap-4 mb-6 pb-5 border-b border-white/5">
```

ml/train_baseline.py (Lines 1 - 56)

```python
import os
import json
import pandas as pd
import numpy as np
import joblib
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix
)

BASE_DIR = os.path.dirname(__file__)
DATA_DIR = os.path.join(BASE_DIR, "data")
MODELS_DIR = os.path.join(BASE_DIR, "models")
os.makedirs(MODELS_DIR, exist_ok=True)

TRAIN_PATH = os.path.join(DATA_DIR, "train.csv")
TEST_PATH = os.path.join(DATA_DIR, "test.csv")
MODEL_OUTPUT_PATH = os.path.join(MODELS_DIR, "baseline_model.joblib")
METRICS_OUTPUT_PATH = os.path.join(MODELS_DIR, "baseline_metrics.json")

def train_and_evaluate():
    if not os.path.exists(TRAIN_PATH) or not os.path.exists(TEST_PATH):
        raise FileNotFoundError("Train or test CSV files missing! Run data_loader.py first.")

    print("Loading train and test datasets for baseline training...")
    train_df = pd.read_csv(TRAIN_PATH)
    test_df = pd.read_csv(TEST_PATH)

    train_texts = train_df['cleaned_text'].fillna('').tolist()
    train_labels = train_df['binary_label'].values

    test_texts = test_df['cleaned_text'].fillna('').tolist()
    test_labels = test_df['binary_label'].values

    print(f"Training set size: {len(train_texts)}, Test set size: {len(test_texts)}")

    print("Fitting TF-IDF Vectorizer (max_features=5000, ngram_range=(1, 2))...")
    vectorizer = TfidfVectorizer(max_features=5000, ngram_range=(1, 2))
    X_train = vectorizer.fit_transform(train_texts)
    X_test = vectorizer.transform(test_texts)

    print("Training Logistic Regression model...")
    clf = LogisticRegression(max_iter=1000, random_state=42)
    clf.fit(X_train, train_labels)

    print("Evaluating baseline model on held-out test set...")
    preds = clf.predict(X_test)
    probs = clf.predict_proba(X_test)[:, 1]
```

deploy_cloudshell.sh (Lines 1 - 40)

```
#!/bin/bash
set -e

echo "============================================================"
echo "   TrustLens XAI - Zero-Cost CloudShell Deployment         "
echo "============================================================"

echo "[1/4] Building production React frontend bundle..."
cd frontend
npm install --silent
npm run build
cd ..

echo "[2/4] Building SAM serverless package..."
sam build

echo "[3/4] Deploying SAM stack to AWS..."
sam deploy \
  --stack-name trustlens-xai \
  --resolve-s3 \
  --capabilities CAPABILITY_IAM \
  --no-confirm-changeset

echo "[4/4] Uploading frontend to S3..."
BUCKET_NAME=$(aws cloudformation describe-stacks --stack-name trustlens-xai --query "Stacks[0].Outputs[?OutputKey=='S3BucketName'].OutputValue" --output text)
CLOUDFRONT_URL=$(aws cloudformation describe-stacks --stack-name trustlens-xai --query "Stacks[0].Outputs[?OutputKey=='CloudFrontUrl'].OutputValue" --output text)
API_URL=$(aws cloudformation describe-stacks --stack-name trustlens-xai --query "Stacks[0].Outputs[?OutputKey=='ApiUrl'].OutputValue" --output text)

cd frontend
VITE_BACKEND_URL=$API_URL npm run build
aws s3 sync dist/ s3://$BUCKET_NAME --delete
cd ..

echo "============================================================"
echo "             DEPLOYMENT COMPLETED SUCCESSFULLY!            "
echo "============================================================"
echo "API Gateway URL : $API_URL"
echo "Live Web App    : $CLOUDFRONT_URL"
echo "Storage (DB)    : Amazon DynamoDB (On-Demand / 0.00 USD base)"
echo "============================================================"
```